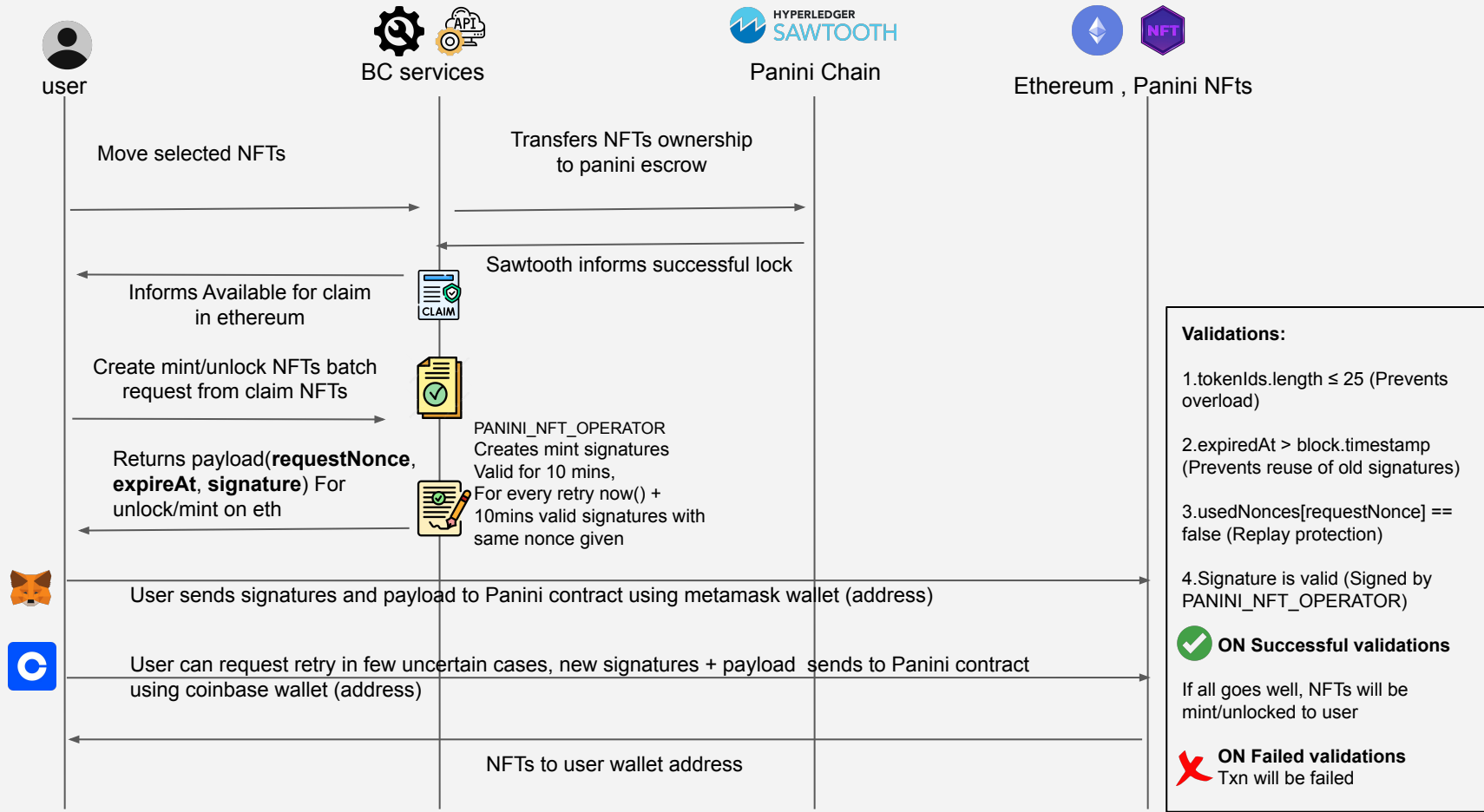


[Bridging & Smart Contracts Overview

Smart Contract – Bridge & Whitelist Marketplaces

1. **Contract Ownership & Role Control**
 - Owner holds full contract ownership, can upgrade contracts, assign/revoke roles (like PANINI_NFT_OPERATOR, WHITELISTED_MARKETPLACE), and manage other key permissions.
2. **Signature-Based Minting & Unlocking**
 - PANINI_NFT_OPERATOR signs ECDSA-based authorizations for minting or unlocking NFTs, enabling secure, verifiable one-time operations across chains.
3. **Bridge Locking to Escrow (Ethereum)**
 - NFTs moving to Sawtooth are locked in an escrow (same contract takes ownership) on Ethereum, ensuring safe custody and transfer freeze until unlock.
4. **Unlocking NFTs in Sawtooth**
 - Upon bridge validation, NFTs will be unlocked on Sawtooth, completing the escrow-to-bridge transfer cycle.
5. **Whitelisted Marketplaces Enforcement**
 - Only approved marketplaces are allowed for transfers and approvals, ensuring royalty enforcement and compliance across the ecosystem.
6. **One-Chain NFT Existence Model**
 - NFTs are designed to exist only on one chain at a time, maintaining uniqueness and preventing cross-chain duplication.
7. On Ownership control, Multisig Wallet([safeWallet](#)) is preferred.

Bridge : Mint Or Unlock (1/2)



Bridge : Mint Or Unlock (2/2)

Note : Assuming All sawtooth side locks are done and ready for Mint/Unlock

- Once ready, User request for claim to unlock NFTs on ethereum.
- PANINI_NFT_OPERATOR signs a payload (chainId,requestNonce, tokenIds, tokenURIs, userWalletAddress, expireAt = now + 10 mins). The signature is valid for 10 minutes.
- If the user retries (e.g., due to wallet issues), they get a new signature with extended expiry, but the requestNonce stays the same.
- User (using metamask) send txn to smart contract function on ethereum.
- In smart contracts,
 - Input Validation:
 - Ensure tokenIds.length === tokenURIs.length and total ≤ 25 .
 - Confirm expireAt is in the future.
 - Ensure requestNonce hasn't been used (enforce one-time use).
 - Signature Verification:
 - Reconstruct the signed message using chainId, requestNonce, tokenIds, tokenURIs, userWalletAddress, and expireAt.
 - Validate the signature is from PANINI_NFT_OPERATOR.
 - Mint or Unlock Logic:
 - Loop through each tokenId:
 - Revert if already burned.
 - Mint if token doesn't exist.
 - Transfer if token exists.
 - Nonce-Based Replay Protection:
 - Even if users retry with multiple signatures, only one succeeds per requestNonce; others fail once nonce is marked used.
- On success of all steps user gets NFTs on ethereum.

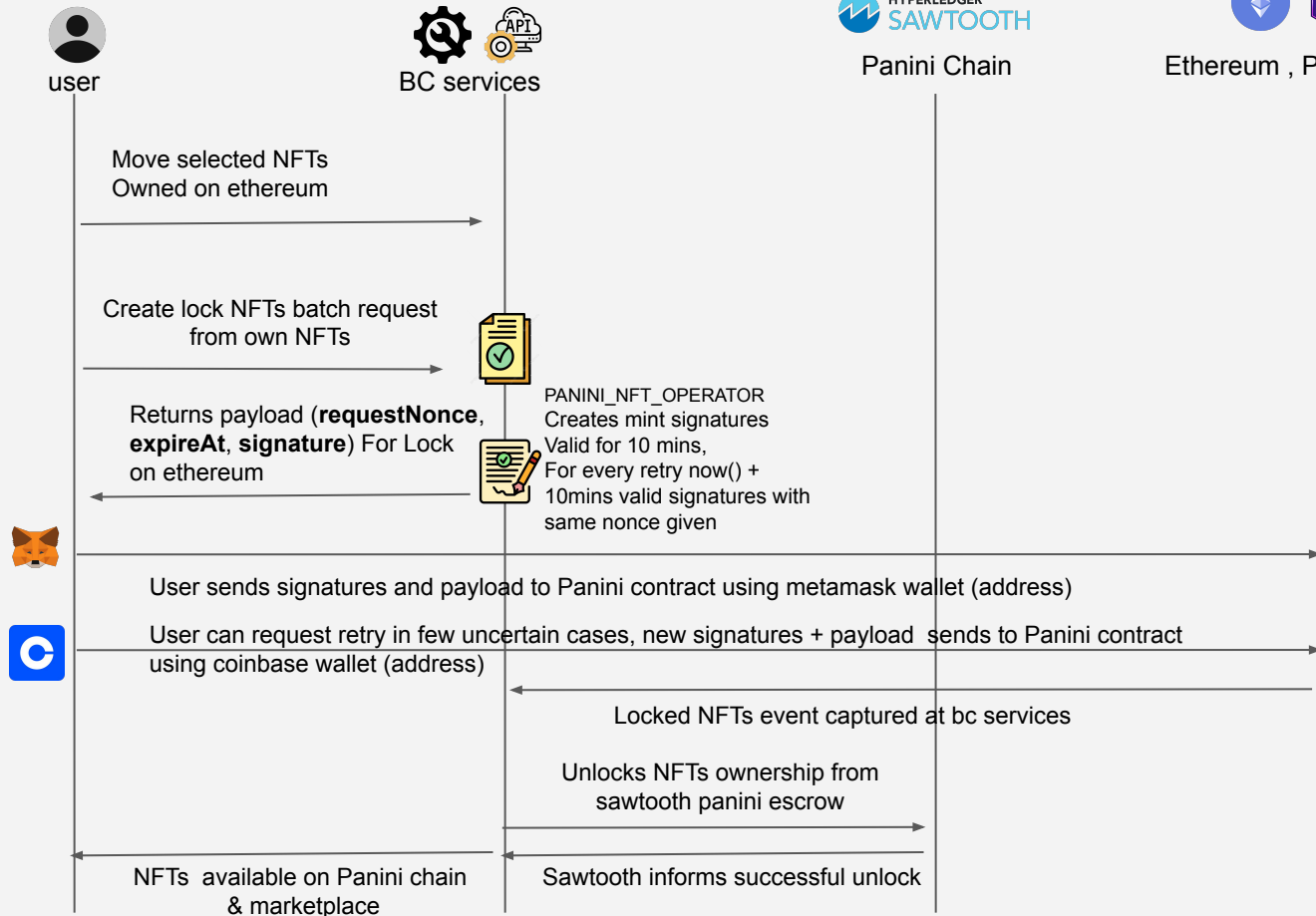
Bridge : Locking (1/2)



Panini Chain



Ethereum , Panini NFTs



Validations:

- 1.tokenIds.length ≤ 25 (Prevents overload)
- 2.expiredAt > block.timestamp (Prevents reuse of old signatures)
- 3.usedNonces[requestNonce] == false (Replay protection)
4. Signature is valid (Signed by PANINI_NFT_OPERATOR)

ON Successful validations

If all goes well, NFTs will be mint/unlocked to user

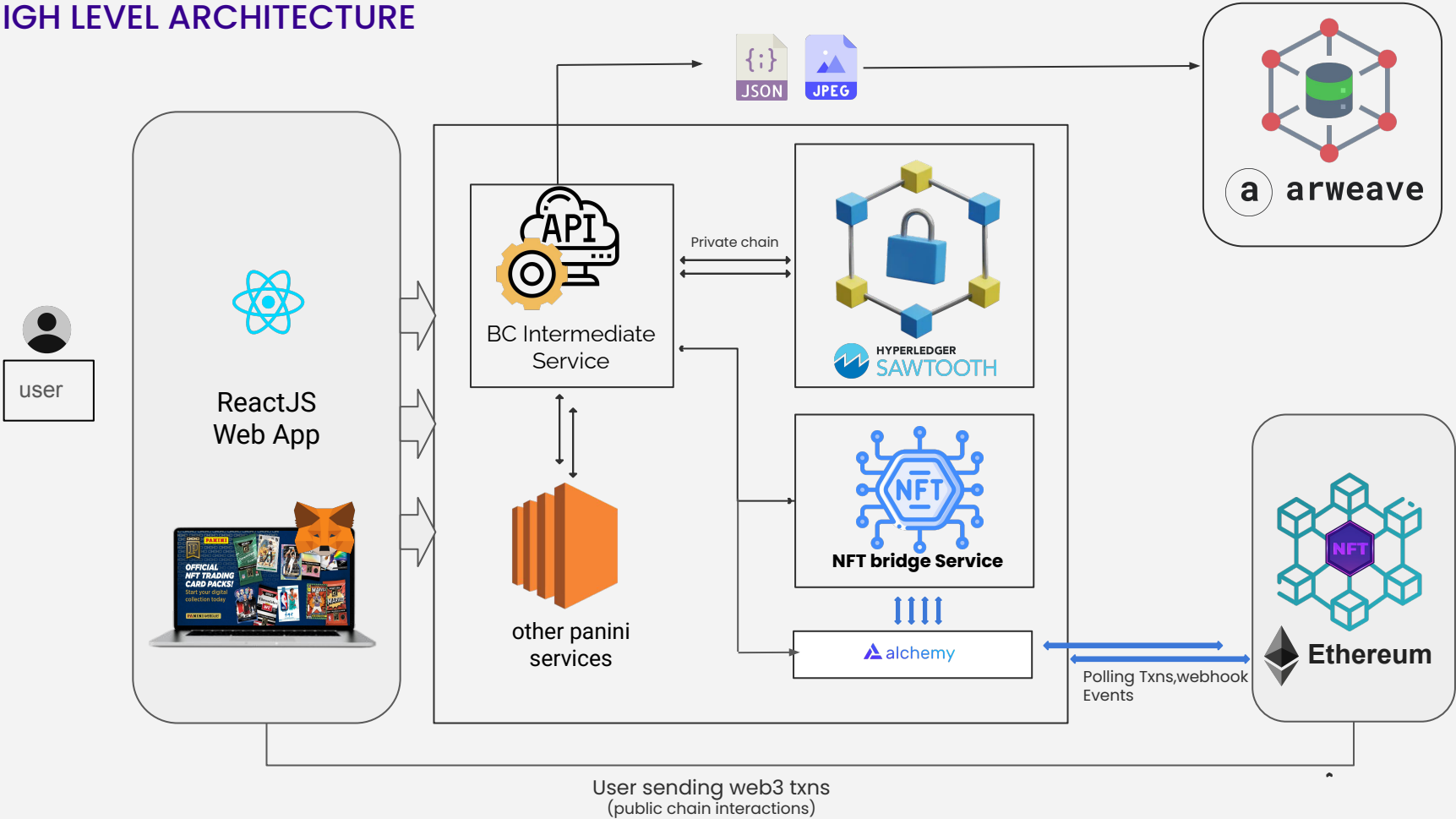
ON Failed validations
Txn will be failed

Bridge : Locking (2/2)

Note : Assuming All ownership checks are done and ready for locking on eth and unlocking in sawtooth

- User can select public chain panini NFTs, he own on ethereum and request for signature to bring it back.
- PANINI_NFT_OPERATOR signs a payload (chainId,requestNonce, tokenIds, userWalletAddress, expireAt = now + 10 mins). The signature is valid for 10 minutes.
- If the user retries (e.g., due to wallet issues), they get a new signature with extended expiry, but the requestNonce stays the same.
- User (using metamask) send txn to smart contract function on ethereum.
- In smart contracts,
 - Input Validation:
 - Ensure tokenIds.length $\leq$ 25.
 - Confirm expireAt is in the future.
 - Ensure requestNonce hasn't been used (enforce one-time use).
 - Signature Verification:
 - Reconstruct the signed message using chainId,requestNonce, tokenIds, userWalletAddress, and expireAt.
 - Validate the signature is from PANINI_NFT_OPERATOR.
 - Bridge Locking:
 - Loop through each tokenId:
 - Revert if user is not owner..
 - Transfer to escrow if token exists.
 - Nonce-Based Replay Protection:
 - Even if users retry with multiple signatures, only one succeeds per requestNonce; others fail once nonce is marked used.
- On success of lock on ethereum to panini escrow, bc service captures event/txn, and unlock NFTs on panini blockchain

HIGH LEVEL ARCHITECTURE



Panini Royalty Vault – Smart Contract

1. **Secure Treasury Vault**

On-chain vault to hold and manage ETH and ERC20 tokens for platform royalties and token swaps.

2. **Role-Based Access Control**

Owner (multisig) has full control — can assign/revoke vaultManager roles and whitelist receiver addresses.

3. **Controlled Asset Swaps**

Vault Managers can swap ETH/ERC20 to USDC via Uniswap; all converted USDC remains securely stored in the vault.

4. **Restricted Withdrawals**

Only Vault Managers can initiate withdrawals, and only to whitelist/pre-approved receiver addresses set by the owner.

5. **Stable Treasury via Uniswap**

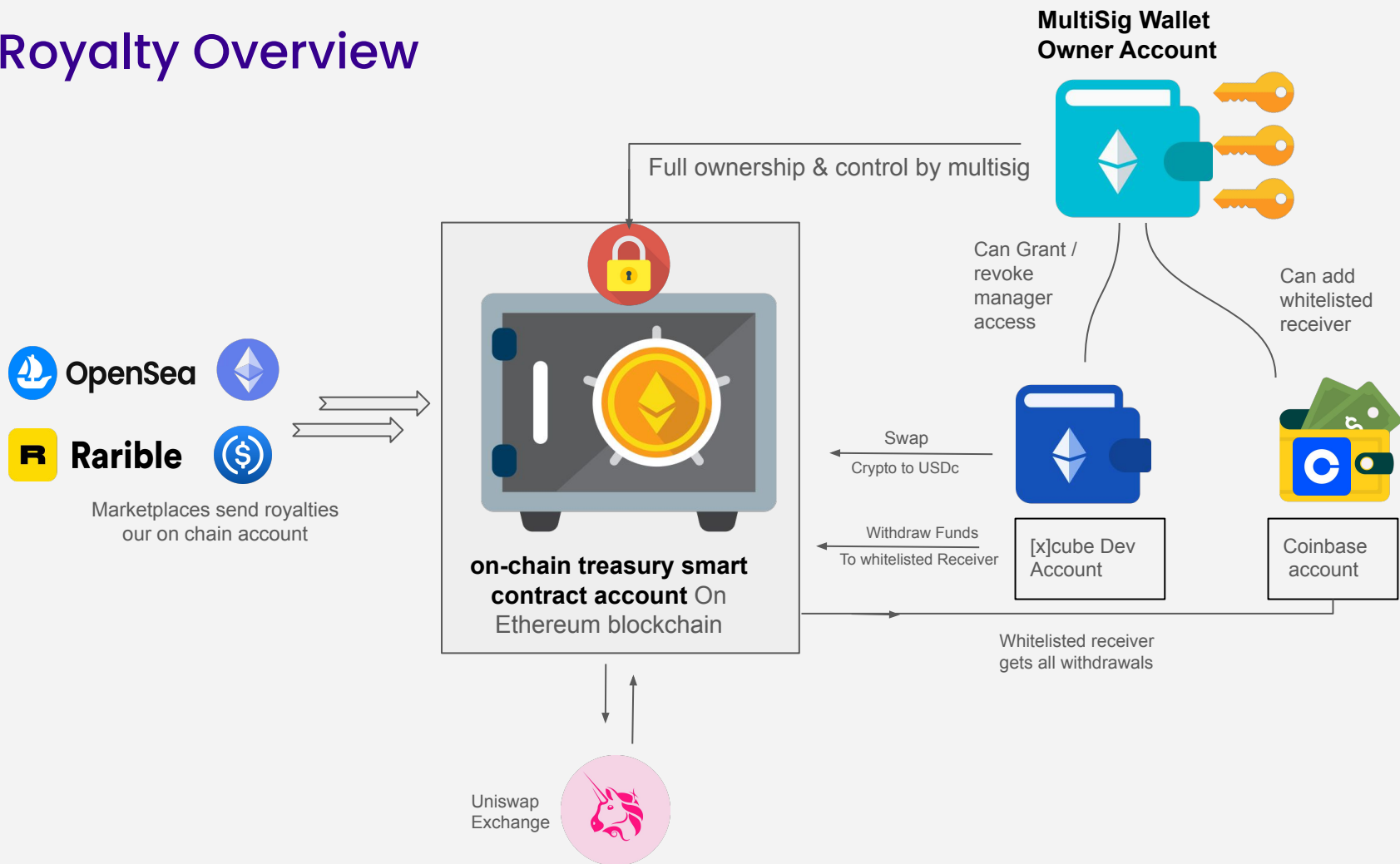
Built-in Uniswap DEX integration enables real-time ETH/token swaps to USDC for maintaining a stable treasury.

6. **Emergency Pause Mechanism**

Owner can pause/unpause all vault operations instantly for emergency response or security control.

7. On Ownership control, Multisig Wallet([safeWallet](#)) might be preferred.

Royalty Overview



Royalty Detailed Flow



OpenSea



Rarible

Marketplaces send royalties
our on chain account



Ethereum blockchain



on-chain treasury account

0xABCCCCxxxxxxxSDF

ETH	WETH	USDC	ERC20
-----	------	------	-------

Withdraw ETH/ ERC20

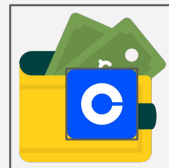
Withdraw USDC

SwapETHtoUSDC

MultiSig Wallet

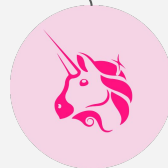


Controlling
using multisig
wallet



Panini Coinbase account
(whitelisted receiver)

ETH / ERC20



USDC

Uniswap Exchange
(Only crypto)



[x]cube Dev



Magento

BC services

THANK YOU!